

Auditoria Forense Final — Geração de Vídeo (PiAPI)

Projeto: Fluxyra (/home/ubuntu/github_repos/saas)

Branch: feat/f3-piapi-landing-pricing

Deploy: https://5be3213ef.abacusai.cloud (systemd fluxyra.service , porta 3000)

Commit desta auditoria: e798aba — fix: handle non-JSON PiAPI responses (502), respect user resolution selection, remove content filters

Data: 26/07/2026

Princípio deste relatório: nenhuma conclusão abaixo é hipótese. Cada causa raiz é sustentada por evidência crua de log (com `requestId` , timestamp e mensagem original) ou por leitura direta do código-fonte (com arquivo e número de linha). Onde algo só pode ser confirmado gerando um vídeo real (o que cria uma task paga na PiAPI), isso está explicitamente marcado como **pendente de teste manual** — esta auditoria **não criou nenhuma task na PiAPI**.

1. Causa raiz do HTTP 502 (resposta não-JSON da PiAPI)

1.1. Evidência de log (crua, não editada)

Duas requisições reais falharam com o mesmo padrão. Ambas foram para o modelo **Seedance** (`task_type: seedance-2`), endpoint POST `https://api.piapi.ai/api/v1/task` .

Requisição 00a64beb :

Timestamp (UTC)	Estágio / evento	Conteúdo relevante
2026-07-26T04:55:08.533Z	api.generate.video / entrada	body do usuário com "resolution": "720p"
2026-07-26T04:55:08.709Z	creditos_debitados	68 créditos debitados
2026-07-26T04:55:08.767Z	piapi.buildVideoPayload / saída	payload construído (ver Seção 2)
2026-07-26T04:55:08.767Z	piapi.submitVideoTask / request	POST https://api.piapi.ai/api/v1/task
2026-07-26T04:55:08.994Z	piapi.submitVideoTask / falha (ms=227)	Unexpected token '<', '<!DOCTYPE "... is not valid JSON
2026-07-26T04:55:09.097Z	piapi_erro_estorno	http_status_devolvido: 502 , creditos_estornados: 68 , generation_id: 9e9f8801-984f-4816-8caa-b879464e864b

Requisição dd1fe259 (padrão idêntico):

Timestamp (UTC)	Evento	Conteúdo
2026-07-26T04:55:23.589Z	entrada	"resolution": "720p"
2026-07-26T04:55:23.971Z	falha (ms=228)	Unexpected token '<', '<!DOCTYPE "... is not valid JSON
2026-07-26T04:55:24.068Z	piapi_erro_estorno	http_status_devolvido: 502

1.2. Diagnóstico (comprovado)

A string Unexpected token '<', '<!DOCTYPE "... is not valid JSON' é o erro que o `res.json()` do JavaScript

lança quando o corpo da resposta **começa com** `<!DOCTYPE`, ou seja, é uma **página HTML**, não JSON.

A PiAPI (`api.piapi.ai`) fica atrás de Cloudflare. Quando está sobrecarregada ou sob challenge, o Cloudflare responde com uma **página HTML** (challenge/erro) em vez do JSON esperado. O código antigo chamava

`await res.json()` diretamente; ao receber HTML, isso disparava um `SyntaxError` que subia até o `catch` da rota,

que devolvia **HTTP 502 com a mensagem crua e ilegível** Unexpected token '<'... para o usuário.

Isto é fato observado nos logs (mensagem literal `<!DOCTYPE`), **não** uma suposição sobre o comportamento da PiAPI.

1.3. Correção aplicada

Arquivo: `src/lib/piapi/client.ts`, função `piapiFetch`, linhas **117-150**.

Antes (comportamento reconstruído a partir do erro de log): chamava `const data = await res.json();` diretamente — qualquer resposta não-JSON explodia com `SyntaxError`.

Depois (código atual, linhas 117-150):

```
const contentType = res.headers.get("content-type") || "";
const rawText = await res.text();
let data: any;
if (contentType.includes("application/json")) {
  try {
    data = JSON.parse(rawText);
  } catch {
    auditLog("piapi.fetch", "parse_error", requestId, { /* url, http_status, content_type, raw_preview */ }, ...);
    throw new PiAPIError(
      `Provider retornou JSON inválido (HTTP ${res.status}). Tente novamente em alguns instantes.`,
      502,
      { raw: rawText.slice(0, 500) }
    );
  }
} else {
  auditLog("piapi.fetch", "non_json_response", requestId, { /* ... raw_preview ... */ }, ...);
  throw new PiAPIError(
    `Provider temporariamente indisponível (HTTP ${res.status}, recebeu ${contentType} || "HTML" em vez de JSON). Tente novamente em alguns instantes.`,
    503,
    { raw: rawText.slice(0, 500) }
  );
}
```

Efeito:

- Resposta HTML (challenge Cloudflare / sobrecarga) → **HTTP 503** com mensagem legível “Provider temporariamente indisponível...” + o HTML cru registrado no log de auditoria (`non_json_response`) para diagnóstico.
 - Header diz `application/json` mas corpo é inválido → **HTTP 502** com “Provider retornou JSON inválido...”.
 - O `SyntaxError` cru `Unexpected token '<'` **nunca mais** chega ao usuário.
-

2. Bug da resolução ignorada (720p selecionado → 1080p enviado)

2.1. Evidência de log (crua)

Na requisição 00a64beb :

- Evento `api.generate.video` / entrada @ 04:55:08.533Z : o body do usuário continha `"resolution": "720p"` .
- Evento `piapi.buildVideoPayload` / saída @ 04:55:08.767Z : o payload efetivamente montado continha `"resolution": "1080p"` .

Idêntico em dd1fe259 (entrada 720p @ 04:55:23.589Z → saída 1080p @ 04:55:23.742Z).

Ou seja: **o usuário pediu 720p e o sistema enviou 1080p à PiAPI**, comprovado pela comparação entrada×saída no mesmo `requestId` .

2.2. Diagnóstico (comprovado por código)

O construtor de payload em `buildVideoPayload` derivava a resolução **exclusivamente do campo `quality`** ,

ignorando o `resolution` explícito escolhido pelo usuário. Para Seedance a regra antiga era `quality === "high" ? "1080p" : ...` , então o campo `resolution` do body do usuário nunca era usado.

Além disso, o campo `resolution` do body **não estava sendo repassado** de `route.ts` para `buildVideoPayload` .

2.3. Correção aplicada

(a) Repasse do campo — `src/app/api/generate/video/route.ts` , linha 221:

```
resolution: typeof resolution === "string" ? resolution : undefined,
```

(o `resolution` já era desestruturado do body; agora é efetivamente passado a `buildVideoPayload`).

(b) Tipo — `src/lib/piapi/client.ts` , interface `BuildVideoArgs` : adicionado `resolution?: string` (com prioridade sobre a derivada de `quality`).

(c) Prioridade em cada provider — em todos, `args.resolution` passou a ser a **primeira** fonte, com o valor derivado de `quality` apenas como fallback, mantendo os limites técnicos de cada modelo:

Provider	Linha	Regra atual
Seedance	419-421	<code>args.resolution \\ (low→480p / medium→720p / high→1080p) ; se não for seedance-2 , teto 720p</code>
Wan	474-475	<code>(args.resolution \\ high→1080p/ 720p).toUpperCase() ; 480P→720P (Wan usa “P” maiúsculo)</code>
Hailuo	508-511	<code>wants1080 = args.resolution ? inclui "1080" : quality=="high" ; 1080 numérico só com dura- tion===6 , senão 768</code>
Veo3 / Veo3.1	526-527	<code>args.resolution \\ (high→1080p/720p) ; 480p→720p</code>
Kling Omni	580-581	<code>args.resolution \\ (high→1080p/720p) ; 480p→720p</code>

Pendente de teste manual: a confirmação de que uma nova geração real produz `entrada.resolution == saida.resolution` exige criar uma task paga na PiAPI (o usuário faz). Pela leitura de código, a correção está aplicada em todos os providers.

3. Varredura forense de filtros internos de conteúdo

Comando executado nesta auditoria:

```
grep -rniE "real person|deepfake|celebrity|nsfw|content filter|moderate|face detect|auto_upload_assets" src/
```

Resultado — apenas 2 ocorrências, ambas em comentários, nenhuma lógica ativa:

Arquivo:linha	Trecho	Classificação	Ação
src/lib/piapi/cli-ent.ts:58	comentário: "...de falhas (ex.: 'real person', 'content restriction', 'plan limit')"	Comentário explicativo (documenta possíveis mensagens que a PiAPI pode retornar)	Mantido — não é filtro
src/lib/piapi/cli-ent.ts:461	comentário: "NÃO enviamos auto_upload_assets: true ..."	Comentário documentando remoção de workaround não-oficial	Mantido — não é filtro

Conclusão: não há nenhuma lógica interna de classificação/bloqueio de conteúdo (real person, deepfake, celebridade, NSFW, detecção facial, moderação) no código. A responsabilidade por decidir o que pode ou não ser gerado é **exclusivamente do provider (PiAPI)**. As mensagens de erro do provider são preservadas e repassadas ao usuário (ver `status/route.ts`, que apenas prefixa neutro "O provider rejeitou a solicitação:" sem substituir o texto original).

3.1. Validações técnicas legítimas (mantidas)

Estas **não** são filtros de conteúdo — são validações de integridade de arquivo, e foram preservadas:

Local	Regra	Propósito
src/app/api/upload/route.ts:10	MAX_BYTES = 20 * 1024 * 1024 (20 MB)	Limite de tamanho de upload
src/app/api/upload/route.ts:12	ALLOWED_PREFIXES = ["image/", "video/", "audio/"]	Aceitar só mídia (por prefixo MIME)
src/app/api/upload/route.ts:14	EXT_BY_MIME	Mapear extensão correta pelo MIME
nginx (deploy/5be3213ef.conf)	client_max_body_size 25m	Teto de corpo da requisição
Dock (frontend)	accept="image/*,video/*,audio/*"	Seletor de arquivo restrito a mídia

Nenhuma biblioteca de reconhecimento facial ou moderação de conteúdo está presente no repositório.

4. Tabela completa de códigos HTTP

Todos gerados em `src/app/api/generate/video/route.ts` (salvo indicação). Mensagens exatas ao usuário:

HTTP	Onde / evento	Condição	Mensagem ao usuário
400	validacao_falhou_400	prompt ou model_uuid ausente	{"error": "Prompt e modelo são obrigatórios"}
401	auth_falhou_401	Usuário não autenticado	{"error": "Não autorizado"}
402	creditos_insuficientes_402	Créditos do usuário no app insuficientes	{"error": "Créditos insuficientes", "required", "available"}
402	catch (Fix 3, linhas 284-297)	PiAPI retorna “insufficient credits” / “freeze credit” / “quota not enough” / “account point” (créditos da conta PiAPI)	Mensagem original do provider preservada
403	plano_insuficiente_403	Plano do usuário não cobre o recurso	{"error": ...} do plano
404	modelo_nao_encontrado_404	model_uuid não existe no catálogo	{"error": "Modelo não encontrado"}
500	insert_generation_falhou_500	Falha ao inserir a geração no banco	{"error": "Erro interno do servidor"}
500	excecao_500	Exceção não prevista	{"error": "Erro interno do servidor"}
502	catch (linhas 296-300)	PiAPI retornou JSON inválido (header JSON, corpo quebrado) ou erro genérico do provider	Mensagem do provider preservada / “Provider retornou JSON inválido...”
503	catch (Fix 3, linhas 291-299)	PiAPI retornou não-JSON (HTML de challenge Cloudflare / sobrecarga)	“Provider temporariamente indisponível (HTTP ..., recebeu HTML em vez de JSON). Tente nova-

HTTP	Onde / evento	Condição	Mensagem ao usuário
			mente em alguns instantes.”

Distinção 502 × 503 (Fix 3, `route.ts` linhas 283–300): o catch inspeciona a mensagem em minúsculas —

`temporariamente indisponível` / `recebeu html` / `em vez de json` / `non_json` → **503** (temporário, pode retentar); `insufficient credits` etc. → **402**; qualquer outro erro do provider → **502**.

5. Estado atual e itens pendentes de teste manual

Verificado nesta auditoria (com evidência):

- ✓ Build TypeScript sem erros (`npx tsc --noEmit rc=0; npm run build rc=0`).
- ✓ Serviço reiniciado e ativo (`systemctl is-active fluxyra.service` → `active`).
- ✓ Health check `GET /` → HTTP 200; `GET /api/models?type=video` → HTTP 200.
- ✓ Varredura de filtros internos: nenhuma lógica ativa de classificação de conteúdo (Seção 3).
- ✓ Código das correções presente e conferido linha a linha (Seções 1–2).
- ✓ Commit `e798aba` registrado.

Pendente de teste manual (requer criar task real/paga na PiAPI — feito pelo usuário):

- ⌚ Confirmar, numa geração real, que `entrada.resolution == saida.resolution` no payload (ex.: 720p → 720p).
- ⌚ Confirmar que, em caso de resposta HTML da PiAPI, o usuário recebe **HTTP 503** com a mensagem legível “Provider temporariamente indisponível...” (em vez do antigo `Unexpected token '<'`).
- ⌚ Confirmar que a mensagem de rejeição de conteúdo do provider (quando houver) chega ao usuário na íntegra.

Nota de integridade: esta auditoria **não criou nenhuma task na PiAPI** e não gerou nenhum vídeo. Todas as evidências de falha (`00a64beb` , `dd1fe259`) vêm de execuções de diagnóstico **anteriores** registradas em `logs/app.log` . As três confirmações acima só podem ser obtidas com uma geração real e por isso ficam a cargo do usuário.

6. Resumo das alterações (commit e798aba)

Arquivo	Mudança
src/lib/piapi/client.ts	Fix 1 (Content-Type check → 502/503 com mensagem legível); Fix 2 (args.resolution prioritário em Seedance/Wan/Hailuo/Veo3/Kling Omni + campo no BuildVideoArgs)
src/app/api/generate/video/route.ts	Repasse de resolution ao buildVideoPayload (l.221); Fix 3 (classificação 402/503/502 no catch, l.283-300)

Restrições respeitadas: nenhum filtro de conteúdo próprio foi implementado; nenhuma mensagem de erro do provider foi substituída por texto hardcoded (apenas mensagens de infraestrutura — JSON inválido / provider indisponível — foram tornadas legíveis); nenhuma task foi criada na PiAPI.